أمان

Aman — Food Safety Transparency Platform

Web Platform Build Specification

How the System Works & What to Build

Document	Build spec v1.0 — engineering reference
Audience	Development team (frontend, backend, design)
Scope	MVP — Nablus pilot, food establishments
Target	Working demo in 4 weeks
Status	Not yet implemented — this is the build plan
Date	August 2026

0. How to Use This Document

This is a build spec, not a proposal. Every screen, rule, and data structure needed for the MVP is defined here. If something is not in this document, it is **not** in the MVP.

If you are…	Read these sections first
Backend developer	§3 Roles → §7 Data Model → §8 API → §6 Core Logic
Frontend developer	§4 Site Map → §5 Screen Specs → §10 Design System → §9 Offline
Designer	§5 Screen Specs → §10 Design System
Team lead / presenter	§1 Scope → §12 Build Order → §13 Demo Script

Read this before writing any code

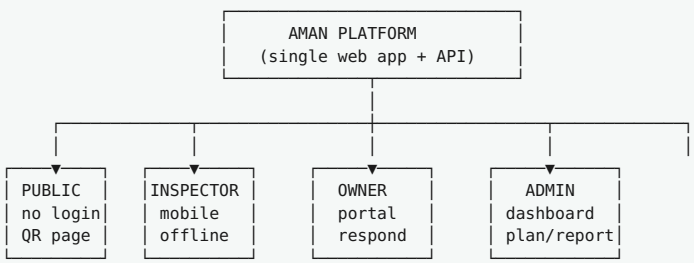
The single most important design rule in this system: **Aman never issues a grade. The municipality does.** Our platform calculates, stores, and displays — the legal authority stays with the inspector and the municipality. Every screen, every API response, every piece of UI copy must reflect this. If a screen makes it look like Aman is the certifying body, the screen is wrong.

0.1 Naming conventions used throughout

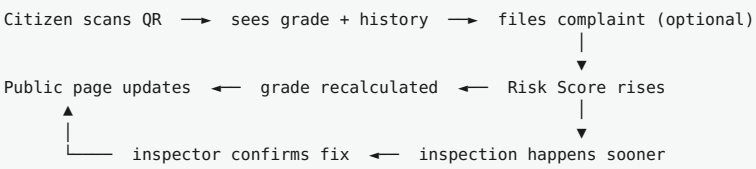
- **Establishment** — any inspected food business (restaurant, bakery, butcher, café). Never hardcode "restaurant".
- **Inspection** — one completed visit producing a score and grade.
- **Violation** — one failed checklist item within an inspection.
- **Complaint** — a citizen report. A complaint never changes a grade directly.
- **Risk Score** — 0–100 priority number that orders the inspection queue.

1. What We Are Building

One codebase, one database, four different views depending on who is logged in. It is **not** four separate applications.



1.1 The core loop (this is the product)



Every feature in the MVP exists to close this loop. If a proposed feature does not sit somewhere on this circle, it is out of scope.

1.2 In scope for MVP

Module	What it must do
Public establishment page	Reachable by QR scan without login. Shows grade, last inspection date, open violations count, complaint button.
Complaint intake + tracking	Submit with optional photo, receive reference number, check status later by reference number.
Inspector PWA	Login, prioritized daily list, digital checklist that works offline, photo capture, automatic scoring, submit.
Risk Score engine	Transparent weighted formula, recalculated nightly and on every event, orders the inspection queue.
Owner portal	See violations and recommendations, upload proof of fix, request re-verification.
Admin dashboard	KPIs, establishment registry, complaint queue, inspection planning, basic reports, QR generation.

1.3 Explicitly out of scope (do not build these)

- Native mobile apps — the PWA covers both inspector and citizen.
- Any online payment or subscription billing.
- Machine-learning risk model (the weighted formula is deliberate — see §6.2).
- Any certificate or badge issued under the Aman brand.
- Multi-municipality tenancy, heat maps, computer vision, predictive analytics.
- Public star ratings or free-text reviews (this is a regulatory tool, not Yelp).

2. Technology Stack

Chosen for: one language across the stack, offline capability, cheap hosting, and the ability to self-host locally (digital sovereignty was a stated requirement of the challenge).

Layer	Choice	Why this one
Frontend	React + Vite, TypeScript	Fast build, huge ecosystem, easy PWA setup via `vite-plugin-pwa`.
Styling	Tailwind CSS with RTL enabled	Use logical properties (`ms-` / `me-`, not `ml-` / `mr-`) so RTL works without a second stylesheet.
Backend	Node.js + Express (or NestJS)	Same language as frontend; team ships faster with one mental model.
Database	PostgreSQL	Relational integrity matters here (inspections → violations → evidence). Use JSONB for checklist snapshots.
ORM	Prisma	Type-safe queries, painless migrations, readable schema file.
Auth	JWT (access + refresh), bcrypt	Stateless, works offline-friendly on the inspector app.
File storage	Local disk (MVP) → MinIO later	Photos stay on local infrastructure. No foreign cloud dependency.
Offline store	IndexedDB via Dexie.js	Inspector app must survive zero connectivity inside a basement kitchen.
QR generation	`qrcode` (npm), server-side	Generate printable PNG/SVG sheets from the admin panel.
Charts	Recharts	Admin dashboard only. Keep it to 2–3 charts.
Notifications	WhatsApp/Telegram bot or SMS gateway	Citizens will not install an app. Meet them where they already are.

Repository layout

```
aman/
├── apps/
│   ├── web/          React app (all four role views)
│   └── api/          Express API
├── packages/
│   └── shared/       Types, grade + risk logic, validation schemas
├── prisma/           schema.prisma, migrations, seed.ts
└── docs/             this spec, checklist definitions
```

Put the grading and risk formulas in `packages/shared` so the same code runs on the server and inside the offline inspector app. Never duplicate that logic.

3. User Roles & Permissions

Role	Login?	Device	Can do
Public مواطن	No	Phone browser	View establishment pages, submit complaints, track own complaint by reference number.
Inspector مفتش	Yes	Phone (offline)	View assigned queue, conduct inspections, capture photos, verify fixes. Cannot edit past inspections or delete records.
Owner صاحب منشأة	Yes	Phone / desktop	View own establishment only, read violations, upload fix evidence, submit a written response.
Admin إدارة البلدية	Yes	Desktop	Everything: registry, assignments, complaint triage, reports, QR generation, checklist and weight configuration.

3.1 Permission matrix

Action	Public	Inspector	Owner	Admin
View public establishment page	✔	✔	✔	✔
Submit complaint	✔	—	—	✔
See complainant identity	✘	✘	✘	✔
Create inspection	✘	✔	✘	✘
Change a grade	✘	✔ (via inspection only)	✘	✘
Upload fix evidence	✘	✘	✔ (own only)	✘
Verify a fix	✘	✔	✘	✘
Edit risk weights / checklist	✘	✘	✘	✔
Generate QR codes	✘	✘	✘	✔

Two hard rules to enforce in code, not just in UI

- **Admins cannot change grades.** Only a submitted inspection changes a grade. This is what protects the system from political pressure — and it is your best answer when a judge asks about integrity.
- **Complainant identity is never exposed to the establishment**, not even indirectly through timestamps or photo EXIF data. Strip EXIF on upload.

4. Site Map & Routes

```
PUBLIC (no auth)
  /                          Home — search + directory
  /e/:slug                   Establishment public page   ← QR CODE TARGET
  /e/:slug/complaint         Complaint form
  /complaint/track           Track by reference number
  /complaint/:ref            Complaint status view
  /about                     How grading works (trust page)

INSPECTOR  (role: INSPECTOR)
  /app/login
  /app/today                 Prioritized queue for today
  /app/establishment/:id     Establishment file + history
  /app/inspect/:id           Checklist (offline-capable)
  /app/inspect/:id/review    Review + sign + submit
  /app/verify/:violationId   Fix verification visit
  /app/sync                  Pending offline items

OWNER  (role: OWNER)
  /portal/login
  /portal                    My establishment overview
  /portal/violations/:id     Violation detail + upload fix
  /portal/history            Past inspections

ADMIN  (role: ADMIN)
  /admin                     KPI dashboard
  /admin/establishments      Registry table
  /admin/establishments/:id  Establishment detail
  /admin/complaints          Triage queue
  /admin/planning            Risk-ranked inspection planning
  /admin/reports             Reports + export
  /admin/qr                  QR generation / print sheets
  /admin/settings            Checklist editor, risk weights, users
```

Slug rule

Public URLs use a readable slug, not a database ID: `/e/golden-oven-nablus`, not `/e/47`. Sequential IDs let anyone enumerate every establishment and scrape the whole registry. Generate the slug from the name plus a short random suffix, and never change it once printed on a QR sticker.

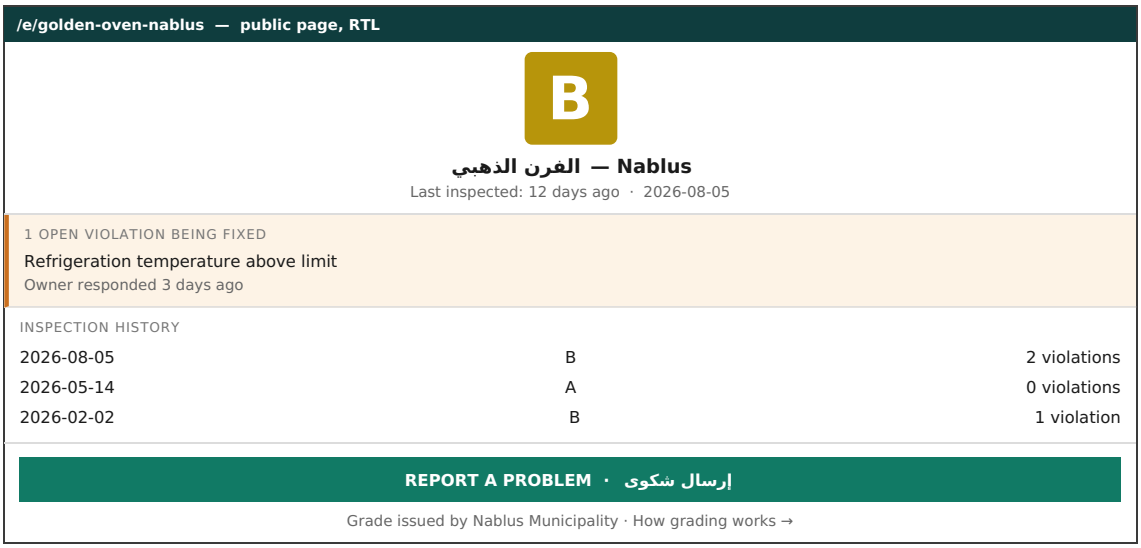
5. Screen Specifications

Each screen below defines: purpose, who sees it, layout, and behaviour. Wireframes are structural only — visual design follows §10.

5.1 PUBLIC — Establishment Page MOST IMPORTANT SCREEN

Route: `/e/:slug` · **Auth:** none · **Device:** phone, one-handed, standing outside a restaurant

Purpose: A citizen has just scanned the QR sticker on the door. They will spend 4–8 seconds here before deciding to walk in or walk away. The grade must be legible from arm's length, instantly.



Behaviour rules

- Grade block is the largest element on the page. Colour and letter together — never colour alone (colour-blind users, and grayscale printing).
- "Last inspected" shows relative time in Arabic ("12 days ago"), with the exact date underneath in small text.
- Open violations show the **category** only, never the raw inspector notes. Notes may contain personal details.
- If the owner has responded, say so. This is the "partnership not punishment" principle made visible — it materially changes how owners feel about the system.
- Attribution line is mandatory on every public page: the grade is issued by the municipality.
- History shows the last 5 inspections maximum, newest first.

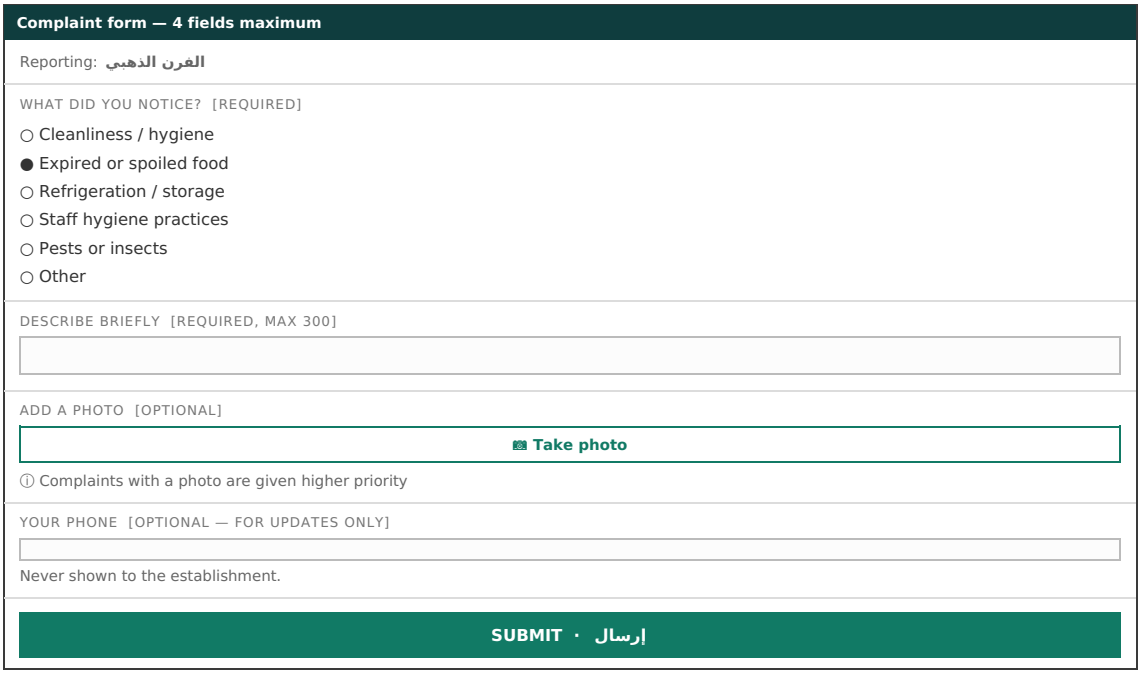
Empty and edge states — specify these now, they are half the work

State	What to show
Never inspected	Grey badge showing "—" with a "not yet inspected" label. Never show a fake A for an uninspected establishment.
Inspection older than 12 months	Show the grade but add a muted note: "This grade may be out of date."
Establishment closed / suspended	Red banner replacing the grade. No complaint button.
Slug not found	Friendly page: "This code is not registered" + link to report a fake sticker.
Grade dropped since last visit	No special treatment. Do not editorialize.

5.2 PUBLIC — Complaint Form

Route: `/e/:slug/complaint` · **Auth:** none

Purpose: Capture a usable report in under 60 seconds. Every extra field loses roughly a third of submissions — keep it brutally short.



Behaviour rules

- **Photo is optional but incentivized.** The hint text is doing real work — it is how the anti-malicious-complaint weighting becomes visible to users.
- Strip EXIF metadata from every uploaded photo on the server. GPS coordinates in a photo can identify the complainant.
- Rate limit: max 3 complaints per IP per establishment per 24h, and max 10 per IP per day overall.
- Add an invisible honeypot field. Do not use CAPTCHA — it kills completion rates on mobile.
- On success, show the reference number **large and copyable**, e.g. `#4821`, plus a "save this number" instruction.

Success screen

```
✓  Complaint received
   Reference: #4821          [ Copy ]
   Status: Under review
   Track it any time at:  aman.ps/complaint/track
```

5.3 PUBLIC — Complaint Tracking

Route: `/complaint/track` → `/complaint/:ref`

Input: reference number only (no login). Show a vertical stepper — this single screen is what separates Aman from every complaint box that citizens have stopped trusting.

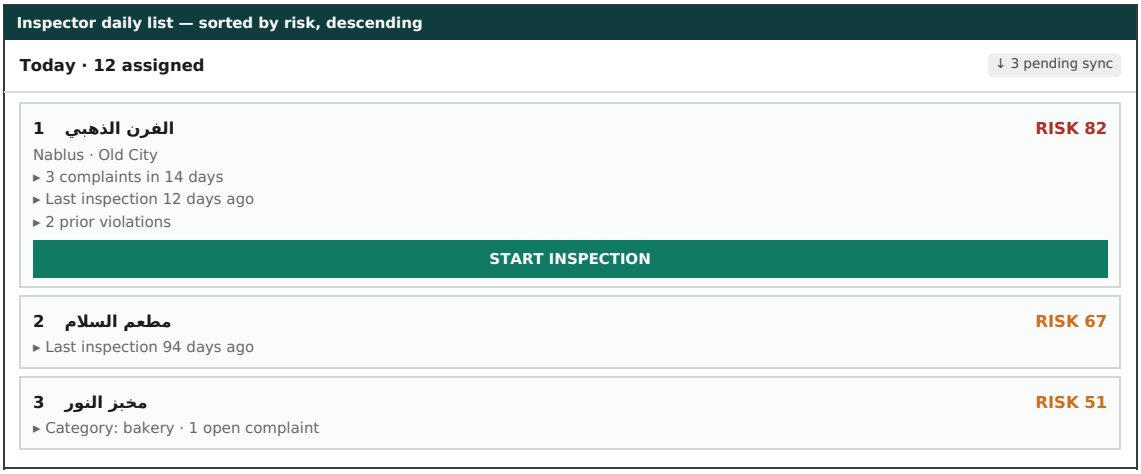
```
●  Received              2026-08-05  ✓
●  Under review          2026-08-06  ✓
●  Inspection scheduled  2026-08-08  ✓
○  Inspected             pending
○  Closed                pending
```

Never expose the inspector's name or internal notes. Show status and dates only.

5.4 INSPECTOR — Today's Queue

Route: `/app/today` · **Auth:** INSPECTOR · **Must work fully offline**

Purpose: Answer one question — where do I go first? This screen is the visible output of the Risk Score, and it is what you demo to judges to prove the system is data-driven.



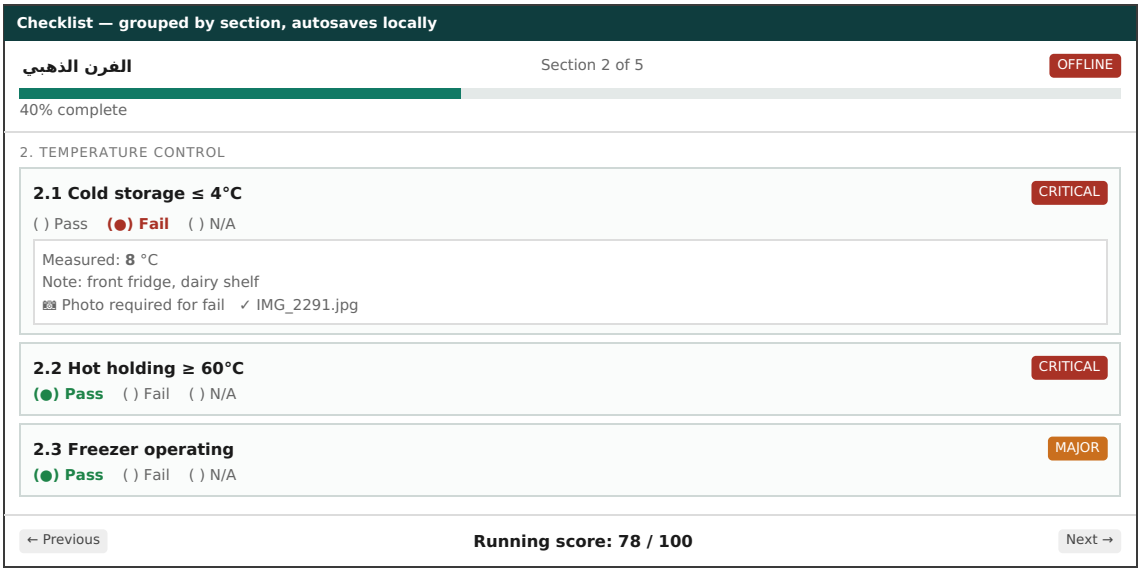
Behaviour rules

- **Always show why.** The 2–3 bullet reasons under each entry are not decoration — they are the explainability requirement. An inspector must be able to justify the visit order, and an owner must be able to understand it.
- Risk band colours: 70+ red, 40–69 amber, below 40 green.
- Pull to refresh re-fetches and re-sorts. When offline, show the cached list with a "last updated" timestamp.
- Tapping Start downloads the full establishment record into IndexedDB before opening the checklist, so losing signal mid-inspection is harmless.

5.5 INSPECTOR — Inspection Checklist CORE SCREEN

Route: `/app/inspect/:id` · **Offline required**

Purpose: Replace the paper form. Must be faster than paper or inspectors will quietly go back to paper — this is the single biggest adoption risk in the whole project.



Behaviour rules

- Default state of every item is **unanswered**, never "Pass". Pre-checked forms produce fake inspections.
- A **Fail on a CRITICAL item requires a photo** — block progress until one is attached. This is the evidence trail that protects the inspector from accusations of bias.
- Autosave to IndexedDB on every single interaction. Assume the phone dies at any moment.
- Running score updates live so the inspector is never surprised at the end.
- N/A must be a real option (a bakery has no hot holding) and N/A items are excluded from the denominator — see §6.1.
- Sections are collapsible; remember which section the inspector was on if the app is closed.

Checklist structure for MVP (5 sections, ~25 items)

#	Section	Items	Example items
1	Personal hygiene النظافة الشخصية	5	Valid health cards, clean uniforms, handwashing station stocked, no bare-hand contact with ready food.
2	Temperature control ضبط الحرارة	4	Cold storage ≤4°C, hot holding ≥60°C, freezer operating, thermometer present.
3	Storage & expiry التخزين والصلاحية	5	No expired product, raw/cooked separation, food off the floor, labelled containers.
4	Premises cleanliness نظافة المكان	6	Surfaces, floors and drains, waste handling, ventilation, restrooms.
5	Pest control مكافحة الحشرات	5	No live pests, no droppings, entry points sealed, pest control contract on file.

Store checklist definitions in the database, not in code — the municipality will want to change wording, and you must not need a deployment to do it. Version every change so old inspections still render against the checklist that was actually used.

5.6 INSPECTOR — Review & Submit

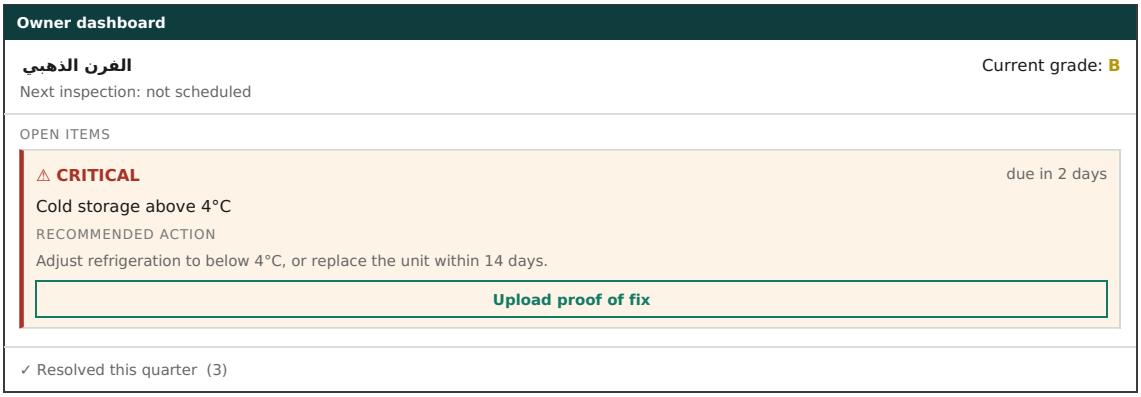
Route: `/app/inspect/:id/review`

- Summary of all failed items with their severity and attached photos.
- Computed score and resulting grade, shown before submission with the previous grade next to it (`B → C`).
- Auto-generated improvement recommendation per violation, editable by the inspector (see §6.5).
- Deadline per violation: critical 48h, major 14 days, minor 30 days — pre-filled, adjustable.
- Inspector signature (canvas) + optional owner acknowledgement signature.
- **Submission is final.** No editing afterwards. Corrections require a new inspection with a reason. Say this in the confirm dialog.
- If offline: queue the submission, show it in `/app/sync`, and retry automatically on reconnect.

5.7 OWNER — Portal

Routes: `/portal`, `/portal/violations/:id`

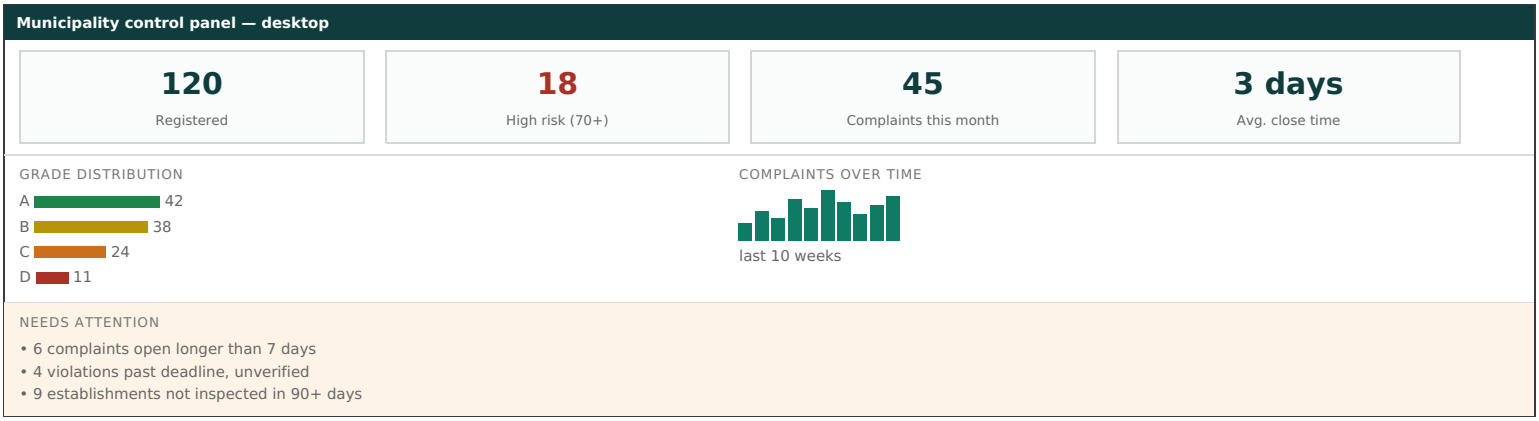
Purpose: Turn a punishment into a to-do list. If owners experience this portal as helpful, adoption stops being a fight.



- Upload proof: photo plus an optional invoice, plus a short note. Status moves to Awaiting verification.
- The owner's response appears on the public page immediately as "owner responded" — but the **grade does not change until an inspector verifies**.
- Include a formal objection channel: one text field, routed to admin. Due process is what makes the system defensible.

5.8 ADMIN — Dashboard

Route: `/admin` · **Device:** desktop



"Needs attention" is the most valuable block on this screen. A dashboard that only reports numbers gets opened twice and abandoned. A dashboard that tells the user what is falling through the cracks gets opened daily — and daily use is what makes the municipality renew the contract.

5.9 ADMIN — Complaint Triage

Route: `/admin/complaints`

Element	Specification
Table columns	Ref, establishment, category, has photo, age in days, status, assigned inspector.
Default sort	Age descending, with photo-backed complaints pinned above photo-less ones of the same age.
Filters	Status, category, has photo, date range, establishment.
Duplicate detection	Flag complaints on the same establishment, same category, within 72h. Group them visually and count them once in the Risk Score.
Actions	Assign to inspector · Mark duplicate · Reject with reason (audited) · Close with outcome.
Rejection	Requires a reason from a fixed list. Rejections are logged and countable — this is your defence against "they just delete complaints".

5.10 ADMIN — Planning & QR

- **Planning** (`/admin/planning`): full establishment list ranked by Risk Score with the contributing factors broken out per row. Select several, assign to an inspector, set a date — this generates their `/app/today` queue.
- **QR** (`/admin/qr`): generate a print sheet (A4, 6 stickers per page) with the establishment name in Arabic, the QR code, the municipality logo, and the short URL underneath for people who cannot scan. Include a batch mode for the whole registry.
- **Settings** (`/admin/settings`): edit checklist items and severities, adjust risk weights (must sum to 100, validate this), manage users. **Every weight change is logged with who and when** — silent tuning of the formula would destroy the system's credibility.

6. Core Logic — The Algorithms

This section is the heart of the system. Implement it in `packages/shared` and unit-test it before building any UI.

6.1 Inspection score and grade

Every checklist item has a severity, and each severity carries a point weight. The score is the percentage of available points earned.

Severity	Points	Meaning
CRITICAL	10	Direct, immediate risk of foodborne illness. Temperature abuse, expired product in service, live pests.
MAJOR	5	Likely to lead to contamination if uncorrected. Poor separation, missing thermometer.
MINOR	2	Good practice and documentation. Missing logs, cosmetic maintenance.

```
function calculateScore(items):
    earned    = 0
    available = 0

    for item in items:
        if item.result == "NA":
            continue                    # excluded from denominator entirely
        available += item.points
        if item.result == "PASS":
            earned += item.points

    if available == 0: return null       # nothing applicable — invalid inspection

    score = round( (earned / available) * 100 )

    # Critical override: any critical failure caps the grade at C
    criticalFails = count(items where severity == "CRITICAL" and result == "FAIL")
    if criticalFails >= 1: score = min(score, 79)
    if criticalFails >= 3: score = min(score, 59)

    return score
```

Grade	Score	Colour	Public meaning
A	90–100	Green #1e8449	Meets standards
B	80–89	Yellow #b7950b	Minor issues found
C	60–79	Orange #ca6f1e	Significant issues, follow-up required
D	below 60	Red #a93226	Serious issues, urgent action required

Why the critical override matters

Without it, an establishment could fail cold storage — the single most dangerous violation in food service — and still score 91% because everything else passed. A grading system that can award an A to a restaurant storing dairy at 8°C is worse than no system at all, because it launders risk into public confidence. The override is not a detail; it is the difference between a credible instrument and a decorative one.

6.2 Risk Score — inspection prioritization

Range 0–100. Recalculated nightly for all establishments, and immediately on any triggering event (new complaint, completed inspection, deadline passed).

```
RiskScore = 0.40 × PriorViolations
          + 0.30 × ComplaintPressure
          + 0.20 × TimeSinceInspection
          + 0.10 × CategoryRisk

_____________________________________________________________________

PriorViolations  (0–100)   weight 40%
    Last 12 months, weighted by severity, decaying over time:
        raw = Σ over violations of ( severityPoints × timeDecay )
        where severityPoints: CRITICAL=10, MAJOR=5, MINOR=2
              timeDecay = max(0, 1 − monthsAgo / 12)
        normalized = min(100, raw × 4)

ComplaintPressure  (0–100)   weight 30%
    Last 90 days, documented complaints weighted higher:
        raw = (documented × 3) + (undocumented × 1)
        where "documented" = has photo or receipt attached
        Duplicates (same establishment + category within 72h)
        count ONCE.
        normalized = min(100, raw × 8)

TimeSinceInspection  (0–100)   weight 20%
        days = daysSince(lastInspection)   # 365 if never inspected
        normalized = min(100, (days / 180) × 100)

CategoryRisk  (0–100)   weight 10%
        Butcher / raw meat handling ........ 100
        Restaurant with full kitchen ........  80
        Bakery / pastry .....................  60
        Café / beverages ....................  40
        Packaged goods retail ...............  20
_____________________________________________________________________
```

Worked example — الفرن الذهبي

Factor	Input	Norm.	Weight	Contribution
Prior violations	1 critical (2 mo ago), 1 major (5 mo)	66	0.40	26.4
Complaint pressure	3 complaints, 2 with photos	56	0.30	16.8
Time since inspection	12 days	7	0.20	1.4
Category	Bakery	60	0.10	6.0
TOTAL RISK SCORE				**50.6 → 51**

Store the breakdown, not just the number

Persist each factor's contribution alongside the total in `risk_score_snapshots`. Two reasons: the inspector queue must be able to display *why* an establishment ranks where it does, and you will need the history to defend the formula to a judge, an owner, or a court. A number without its derivation is not auditable, and an unauditable number has no place in a regulatory system.

6.3 Complaint lifecycle

```
   SUBMITTED
       │
       ├──► DUPLICATE ──────────────────► CLOSED (merged into original)
       │
       ├──► REJECTED (reason required) ─► CLOSED (logged, auditable)
       │
       ▼
   UNDER_REVIEW ──► ASSIGNED ──► INSPECTED ──► CLOSED
                                     │
                                     └──► if violations found:
                                          creates Violation records
                                          triggers grade recalculation
```

Invariant to enforce at the database level: a complaint can never write to the `grade` field. Only an inspection can. Encode this as an application-level rule and cover it with a test — it is the guarantee that makes malicious complaints structurally harmless.

6.4 Violation lifecycle

```
   OPEN ──► OWNER_RESPONDED ──► VERIFIED ──► CLOSED
     │            (evidence)      (inspector)
     │
     └──► OVERDUE (deadline passed, no response)
             │
             └──► escalates: adds to Risk Score, flags on admin dashboard
```

When a violation is verified as fixed, do **not** silently raise the grade. Recalculate the grade only at the next inspection, or record an explicit "follow-up inspection" that re-scores the affected section. Grades must always trace back to an inspection event.

6.5 Auto-generated recommendations

Each checklist item carries a pre-written recommendation template in the database. On failure, the system fills in the measured value:

```
item:     "Cold storage ≤ 4°C"
measured: 8°C
template: "Adjust refrigeration to below {threshold}°C. Current
           reading: {measured}°C. If the unit cannot hold
           temperature, replace it within {deadline} days."
output:   "Adjust refrigeration to below 4°C. Current reading:
           8°C. If the unit cannot hold temperature, replace it
           within 14 days."
```

This is template substitution, not AI, and that is the right call for the MVP: it is deterministic, reviewable, and cannot hallucinate a regulatory instruction. Add an LLM-drafted summary later if you want, but never let a generated sentence become the legal record without an inspector approving it.

7. Data Model

```
establishments ──1:N──► inspections ──1:N──► inspection_items
      │                       │
      │                       └──1:N──► violations ──1:N──► evidence
      │
      ├──1:N──► complaints ──1:N──► complaint_photos
      ├──1:N──► risk_score_snapshots
      └──1:1──► users (owner)

checklist_versions ──1:N──► checklist_items
users ──1:N──► inspections (as inspector)
audit_log (everything sensitive)
```

7.1 Table definitions

establishments

Field	Type	Notes
id	uuid PK	
slug	text unique	Public URL. Immutable once QR is printed.
name_ar / name_en	text	Arabic is required, English optional.
category	enum	BUTCHER, RESTAURANT, BAKERY, CAFE, RETAIL — drives CategoryRisk.
license_number	text	Municipal license reference.
address, lat, lng	text, float	For routing and future mapping.
owner_user_id	uuid FK null	Null until the owner registers.
current_grade	char(1) null	Denormalized for fast public reads. Null = never inspected.
current_score	int null	
last_inspection_at	timestamp null	
current_risk_score	int	Denormalized, refreshed nightly.
status	enum	ACTIVE, SUSPENDED, CLOSED.

inspections

Field	Type	Notes
id	uuid PK	
establishment_id	uuid FK	
inspector_id	uuid FK	
checklist_version_id	uuid FK	Critical: lets you render old inspections correctly after the checklist changes.
type	enum	ROUTINE, COMPLAINT_DRIVEN, FOLLOW_UP.
triggered_by_complaint_id	uuid FK null	Traceability from complaint to visit.
score	int	
grade	char(1)	
previous_grade	char(1) null	Stored so trend display needs no extra query.
started_at / submitted_at	timestamp	Duration is a useful quality signal — a 90-second inspection is suspicious.
inspector_signature	text	Base64 canvas image.
is_offline_submission	bool	

complaints

Field	Type	Notes
id	uuid PK	
reference	text unique	Short human-friendly code, e.g. `4821`.
establishment_id	uuid FK	
category	enum	Matches the form options.
description	text	Max 300 chars.
has_evidence	bool	Drives the ×3 weighting in the Risk Score.
contact_phone	text null encrypted	Encrypted at rest. Admin-visible only.
ip_hash	text	Hashed, for rate limiting only. Never store raw IPs.
status	enum	See §6.3.
duplicate_of_id	uuid FK null	
rejection_reason	text null	Required when status is REJECTED.

violations

Field	Type	Notes
inspection_id / establishment_id	uuid FK	
checklist_item_id	uuid FK	
severity	enum	CRITICAL, MAJOR, MINOR.
measured_value	text null	e.g. "8°C".
recommendation	text	Generated then inspector-approved.
deadline_at	timestamp	
status	enum	See §6.4.
owner_response / responded_at	text, timestamp	
verified_by / verified_at	uuid FK, timestamp	

audit_log

Append-only. Record: actor, action, entity type, entity id, before/after JSON, timestamp, IP hash. Log at minimum: every grade change, complaint rejection, risk-weight change, user role change, and establishment status change. **No deletes, no updates, ever.** When a judge or an owner challenges the fairness of the system, this table is the answer.

8. API Endpoints

8.1 Public (no auth)

Method	Endpoint	Returns / notes
GET	/api/public/establishments/:slug	Grade, score, last inspection date, open violation categories, last 5 inspections. **Never** inspector names or raw notes.
GET	/api/public/establishments	Search + filter directory. Paginated.
POST	/api/public/complaints	Rate limited. Returns reference number only.
GET	/api/public/complaints/:reference	Status timeline only.

8.2 Inspector

Method	Endpoint	Returns / notes
POST	/api/auth/login	Returns access + refresh tokens.
GET	/api/inspector/queue	Assigned establishments sorted by risk, with factor breakdown.
GET	/api/inspector/establishments/:id/bundle	Everything needed offline: profile, history, active checklist version. Call this on "Start".
POST	/api/inspector/inspections	Submit complete inspection. **Must be idempotent** — accept a client-generated UUID so retries after a flaky connection do not create duplicates.
POST	/api/inspector/violations/:id/verify	Confirm a fix.
POST	/api/uploads	Multipart. Strips EXIF, compresses, returns file id.

8.3 Owner & Admin

Method	Endpoint	Returns / notes
GET	/api/owner/establishment	Scoped to the token's establishment. Reject any id parameter.
POST	/api/owner/violations/:id/respond	Evidence + note.
GET	/api/admin/dashboard	KPI aggregates + "needs attention" lists.
GET	/api/admin/complaints	Filterable, includes complainant contact.
PATCH	/api/admin/complaints/:id	Assign, mark duplicate, reject (reason required).
GET	/api/admin/planning	All establishments ranked by risk with breakdown.
POST	/api/admin/qr/batch	Returns printable PDF sheet.
PUT	/api/admin/settings/risk-weights	Validate sum = 100. Writes to audit log.

Idempotency is not optional here

Inspectors work in basements with no signal. The app will retry submissions. Without an idempotency key, one inspection becomes three, and three grades get written for one visit. Generate a UUID on the client when the inspection starts, send it with the submission, and have the server return the existing record if it has already seen that id.

9. Offline Strategy (Inspector App)

Assume no connectivity during the inspection itself. This is not an edge case — it is the normal condition inside a kitchen or a basement storage room.

```
ONLINE (morning, at the office / on wifi)
    ├─ fetch today's queue          → cache in IndexedDB
    ├─ fetch active checklist       → cache in IndexedDB
    └─ fetch establishment bundles  → cache in IndexedDB

OFFLINE (in the field)
    ├─ read everything from IndexedDB
    ├─ write answers to IndexedDB on every tap
    ├─ store photos as Blobs in IndexedDB
    └─ queue the submission in an outbox

RECONNECT
    ├─ upload photos first, collect file ids
    ├─ submit inspection with idempotency key
    ├─ on success: clear from outbox, mark synced
    └─ on conflict (409): server record wins, show a notice
```

Concern	Handling
Storage limits	Compress photos to max 1600px / ~200KB before storing. Cap at 20 photos per inspection.
Sync visibility	Persistent badge in the header showing pending count. Never sync silently — inspectors must be able to confirm their work was delivered.
Partial inspection	Keep drafts in IndexedDB indefinitely. Resume exactly where the inspector left off, including the current section.
Failed sync	Exponential backoff. After 3 failures, surface a manual "Retry" button in `/app/sync`.
Stale queue	Show "last updated X ago" on the queue when offline so the inspector knows the ranking may have shifted.

10. Design System

10.1 RTL — get this right from day one

- Set `<html dir="rtl" lang="ar">` as the default. Arabic is the primary language, not a translation layer bolted on later.
- Use CSS logical properties throughout: `margin-inline-start` not `margin-left`; in Tailwind use `ms-4` / `me-4`, never `ml-4` / `mr-4`.
- **Numbers, times, and Latin identifiers stay LTR.** Wrap them in `<span dir="ltr">` — otherwise `8°C` and `#4821` render mangled inside Arabic sentences.
- Icons with direction (arrows, back chevrons, progress) must mirror. Icons without direction (camera, warning, checkmark) must not.
- Charts: axis on the right, bars growing leftward, legend right-aligned.

10.2 Colour

Token	Hex	Use
Primary	#117A65	Buttons, active states, headers.
Primary dark	#0F3D3E	Nav bars, headings.
Grade A	#1e8449	Grade badge only.
Grade B	#b7950b	Grade badge only.
Grade C	#ca6f1e	Grade badge only.
Grade D	#a93226	Grade badge only.
Surface	#f6faf9	Card backgrounds.
Text	#1a1a1a / #666	Primary / muted.

Reserve the grade colours exclusively for grades. If the same orange appears on a button somewhere, the grade loses its instant readability.

10.3 Typography & touch targets

- Arabic UI font: **IBM Plex Sans Arabic** or **Cairo** (both free, both render well at small sizes). Self-host the files — do not call Google Fonts, both for sovereignty and for reliability on poor connections.
- Grade badge 96px · page title 24px · body 16px · caption 13px. Never go below 14px on the inspector app — it is used outdoors in sunlight.
- Minimum touch target 44×44px. Inspectors tap while holding a thermometer in the other hand.
- Pass/Fail/N/A must be large segmented buttons, not radio dots.

10.4 Components to build once and reuse

`GradeBadge` (sizes sm/md/xl) · `RiskBadge` (with factor tooltip) · `StatusStepper` · `PhotoCapture` (compress + EXIF strip) · `ChecklistItem` · `EmptyState` · `SyncIndicator` · `ConfirmDialog`

11. Security & Privacy Rules

Area	Rule
Complainant identity	Never exposed to owners or inspectors. Admin-only, encrypted at rest. Strip EXIF from every uploaded photo.
Grade integrity	Only an inspection can write a grade. Enforce in the service layer and cover with a test that fails loudly.
Owner scoping	Owner endpoints resolve the establishment from the JWT, never from a URL parameter. Test that owner A cannot read establishment B.
Audit trail	Append-only. No update or delete permissions on `audit_log`, even for admins.
Rate limiting	Complaints: 3 per IP per establishment per day. Login: 5 attempts then a 15-minute lockout.
Uploads	Validate magic bytes, not the file extension. Max 5MB. Images only. Serve from a separate path with no execution rights.
Data residency	Local hosting, local backups. No third-party analytics, no foreign CDN. This was an explicit requirement of the challenge brief.
Secrets	Environment variables only. Never commit `.env`. Rotate the JWT secret before any public pilot.

12. Build Order — Four Weeks

Sequenced so that something demonstrable exists at the end of every week. Do not build in parallel across layers — finish the loop vertically.

Week	Deliverable	Tasks
Week 1 Foundation	Data + public page live with seeded data	Prisma schema and migrations · seed 15 establishments with inspection history · auth and roles · public establishment page · QR generation · deploy pipeline.
Week 2 Inspection	A full inspection can be completed on a phone	Checklist data model + seed 25 items · inspector login and queue · checklist UI · scoring engine + unit tests · photo capture · review and submit.
Week 3 The loop	Complaint → risk → inspection → public update	Complaint form and tracking · Risk Score engine + tests · queue sorting by risk with reasons · admin complaint triage · owner portal and fix verification.
Week 4 Polish + demo	Demo-ready system	Admin dashboard and charts · offline sync hardening · RTL audit on every screen · empty and error states · seed a realistic demo dataset · rehearse §13 twice end to end.

Priority order if you fall behind

Cut in this order, from the bottom up: **1)** admin charts, **2)** owner portal, **3)** complaint tracking page, **4)** offline sync. **Never cut:** the public QR page, the inspector checklist, and the Risk Score queue. Those three are the demo — everything else is supporting cast.

12.1 Definition of done (per feature)

- Works in RTL with Arabic content, checked on a real phone screen width (375px).
- Empty state, loading state, and error state all implemented.
- Core logic covered by unit tests (scoring and risk especially).
- No console errors. No hardcoded Arabic strings scattered in components — keep them in one place.
- Sensitive actions write to the audit log.

13. Demo Script

Eight minutes, one continuous story, no slide-to-app switching. Use two devices: a phone mirrored on screen for citizen and inspector, a laptop for the admin dashboard.

Min	Scene	What you do and say
0–1	The problem	"120 establishments, a handful of inspectors, and a citizen who knows nothing before walking in."
1–2	Citizen scans	Physically scan a printed QR sticker with the phone. Grade **B** appears. "This exists in Los Angeles, London and Dubai. It does not exist in any Palestinian municipality today."
2–3	Citizen reports	Submit a complaint with a photo. Show the reference number. "This is where every other complaint system stops. Watch what happens next."
3–4	Risk engine	Switch to the admin planning screen. The establishment has moved up the ranking. Show the factor breakdown. "The complaint did not change the grade. It changed the queue."
4–6	Inspector	Open the inspector queue — same establishment now ranked #1 with reasons visible. Run the checklist. Fail cold storage, attach a photo. **Turn off wifi to prove offline works.** Submit. Grade drops B → C.
6–7	Public updates	Refresh the public page. Grade is now C. "Before the inspector left the building."
7–8	Owner + close	Owner portal: recommendation, upload proof, awaiting verification. Close with: "We are not asking for inspection authority. We are giving the authority that already has it the tools of this century."

The one move that wins the room

Turning wifi off mid-inspection and continuing to work is worth more than any slide. Every team will claim their app is field-ready; you will be the only one who proves it live. Rehearse it until it is boring.

13.1 Seed data for the demo

- **15 establishments** with realistic Arabic names across all five categories, spread across grades A–D and one never inspected.
- **الفرن الذهبي** as the hero record: grade B, 2 prior violations, 3 recent complaints (2 with photos), last inspected 12 days ago, risk 82. Every number in the demo should back to this record.
- **45 complaints** across the last 30 days, 6 still open, some deliberately duplicated within 72h so you can demonstrate duplicate detection.
- **Inspection history** going back 12 months so the trend and history blocks are not empty.
- Keep the seed script in `prisma/seed.ts` and make it idempotent so you can reset the demo in one command between rehearsals.

Build the loop first. Everything else is decoration.